SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
   print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Assessment Tool 3

Scenario Question 1 Biomedical stemming error

Answer

I lowercased and tokenized the PubMed 20k RCT training split, then applied NLTK Porter stemming to the target forms. The split contained 4,000,338 alphabetic tokens. Stemming groups infection, infected, and infections as infect, but produces infecti for infectious. It also removes the derivational noun ending -ion from infection, which hides the distinction between a noun and the verb infect. This shows why an inflectional ending such as plural -s or past -ed should not be treated like a derivational ending that can change word class or meaning.

Code

```
from collections import Counter
from pathlib import Path
import re
import sys
from nltk.stem import PorterStemmer

path = Path(sys.argv[1])  # PubMed_20k_RCT.../train.txt
text = path.read_text(encoding="utf-8").lower()
tokens = re.findall(r"[a-z]+", text)
counts = Counter(tokens)
stemmer = PorterStemmer()
targets = ["infect", "infection", "infectious", "infected", "infections"]
print("alphabetic tokens:", len(tokens))
for word in targets:
    print(f"{word:12} count={counts[word]:4} stem={stemmer.stem(word)}")
```

Executed output

```
alphabetic tokens: 4000338
infect       count=   3 stem=infect
infection    count=1303 stem=infect
infectious   count= 107 stem=infecti
infected     count= 406 stem=infect
infections   count= 571 stem=infect
```

Error analysis. Infected and infections include inflectional endings (-ed and plural -s), so normalizing them to a lemma can help retrieval. Infection contains derivational -ion; stripping it loses the noun-forming information. Infectious contains derivational -ious, and the Porter result infecti is an unnatural fragment that fails to join the infect group. Stemming is a heuristic string reduction, not a morphological parse.

Improvement. Keep the original token, a POS-aware lemma, and a separate biomedical concept/family identifier. Normalize known inflections first, preserve derivational forms as linked variants, and review domain-specific rules against a medical lexicon. This keeps searches broad without discarding distinctions needed by researchers.

Dataset: PubMed 20k RCT training file from https://github.com/Franck-Dernoncourt/pubmed-rct. Dataset paper: Dernoncourt and Lee, 2017, https://aclanthology.org/I17-2052/.

Scenario Question 2 Finite-state parser for social media words

Answer

The original parser only checks whether a word ends in s and then removes two characters. It has no prefix transitions, no suffix inventory beyond s, and no orthographic rules. It therefore returns every listed form except smartphones as a singular word; for smartphones it incorrectly returns smartphon. I used the public NLTK Twitter Samples corpus (5,000 positive and 5,000 negative tweets) to check how often the six challenge forms occur, and scored the parser on the six gold analyses given in the question.

Code

```
import re, nltk
from nltk.corpus import twitter_samples
nltk.download("twitter_samples", quiet=True)
words = ["happiest", "unbelievable", "running", "reordering",
         "smartphones", "unreadable"]

def old_parser(w):
    return (w[:-2], "Plural Noun") if w.endswith("s") else (w, "Singular")

def fst_parse(w):
    rest, prefix = w.lower(), ""
```

```
    if rest.startswith(("un", "re")):
        prefix, rest = rest[:2] + "-", rest[2:]
    if rest.endswith("iest"):
        root, suffix, feature = rest[:-4] + "y", "-est", "superlative"
    elif rest.endswith("able"):
        root = rest[:-4]
        if root.endswith("v"):
            root += "e"
        suffix, feature = "-able", "derivational adjective"
    elif rest.endswith("ing"):
        root = rest[:-3]
        if len(root) > 1 and root[-1] == root[-2]:
            root = root[:-1]
        suffix, feature = "-ing", "present participle"
    elif rest.endswith("s"):
        root, suffix, feature = rest[:-1], "-s", "plural noun"
    else:
        root, suffix, feature = rest, "none", "base form"
    return prefix or "none", root, suffix, feature

gold = dict(zip(words, [
    ("none", "happy", "-est"), ("un-", "believe", "-able"),
    ("none", "run", "-ing"), ("re-", "order", "-ing"),
    ("none", "smartphone", "-s"), ("un-", "read", "-able")
]))
tweets = twitter_samples.strings("positive_tweets.json")
tweets += twitter_samples.strings("negative_tweets.json")
corpus = " ".join(tweets).lower()
print("tweets:", len(tweets))
for w in words:
    count = len(re.findall(r"\b" + re.escape(w) + r"\b", corpus))
    print(w, "tweet_count=", count, "old=", old_parser(w),
          "new=", fst_parse(w)[:3])
correct = sum(fst_parse(w)[:3] == gold[w] for w in words)
print("exact analyses:", correct, "/", len(words))
```

Executed output

```
tweets: 10000
happiest tweet_count= 2 old= ("happiest", "Singular") new= ("none", "happy", "-est")
unbelievable tweet_count= 0 old= ("unbelievable", "Singular") new= ("un-", "believe", "-able")
running tweet_count= 13 old= ("running", "Singular") new= ("none", "run", "-ing")
reordering tweet_count= 0 old= ("reordering", "Singular") new= ("re-", "order", "-ing")
smartphones tweet_count= 0 old= ("smartphon", "Plural Noun") new= ("none", "smartphone", "-s")
unreadable tweet_count= 0 old= ("unreadable", "Singular") new= ("un-", "read", "-able")
exact analyses: 6 / 6
```

On the six supplied challenge words, exact parse accuracy rises from 0/6 to 6/6. This small set does not estimate general accuracy; most forms are rare or absent in the 10,000-tweet corpus.

The FST uses optional un- or re- prefix transitions, a root state, suffix transitions (-iest, -able, -ing, -s), and spelling rewrites. With a fixed state graph, parsing is O(n) time and O(1) working space per word. Unrestricted affix sequences can add ambiguity.

Programming Question 6 Correcting the plural noun parser

Answer

The original code removes two letters from every word ending in s. That turns cars into ca and cities into citi. It also misses children because that plural does not end in s. Boxes happens to produce box, but only by coincidence. The replacement applies -ies, -es, and -s transitions, validates candidate roots with WordNet, and uses the WordNet noun lemmatizer for irregular forms.

Code

```
import nltk
from nltk.corpus import wordnet as wn
from nltk.stem import WordNetLemmatizer

nltk.download("wordnet", quiet=True)
wnl = WordNetLemmatizer()

def is_known_noun(word):
    return bool(wn.synsets(word, pos="n"))

def parser(word):
    w = word.lower()
```

```
    if w.endswith("ies") and len(w) > 3:
        candidate, rule = w[:-3] + "y", "plural -ies"
    elif w.endswith(("ches", "shes", "xes", "zes", "sses")):
        candidate, rule = w[:-2], "plural -es"
    elif w.endswith("s") and not w.endswith("ss"):
        candidate, rule = w[:-1], "plural -s"
    else:
        lemma = wnl.lemmatize(w, pos="n")
        if lemma != w:
            return lemma, "irregular plural (WordNet)"
        return w, "singular or unknown"

    if is_known_noun(candidate):
        return candidate, rule
    lemma = wnl.lemmatize(w, pos="n")
    if lemma != w:
        return lemma, "plural (WordNet fallback)"
    return candidate, rule + " (unvalidated)"

for word in ["cars", "boxes", "cities", "children"]:
    print(word, "->", parser(word))
```

Executed output

```
cars -> ("car", "plural -s")
boxes -> ("box", "plural -es")
cities -> ("city", "plural -ies")
children -> ("child", "irregular plural (WordNet)")
```

Against the four supplied examples, the original parser gets 1 of 4 analyses correct (boxes only); the corrected parser gets 4 of 4. This is a targeted example score, not broad English coverage. The finite-state transitions scan each word in O(n) time for length n. WordNet supplies lexical checks and irregular forms. Rule-based parsing still has exceptions such as mouse/mice and many -f/-fe plurals, and a candidate can be ambiguous without context.

Lexicon reference: NLTK WordNet interface, https://www.nltk.org/howto/wordnet.html. WordNet is used here for candidate validation and irregular noun lemmas.

Programming Question 7 Correcting morphological feature extraction

Answer

The original program stems the entire Text string as if it were one word, and then stems the feature names after CountVectorizer has already built X. Changing the names in a separate list does not merge columns or change the matrix, so the classifier would still see redundant unstemmed features. The correction tokenizes and stems each document first, then fits CountVectorizer on those normalized documents.

Code

```
import re
import nltk
from nltk.corpus import twitter_samples
from nltk.stem import PorterStemmer
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from time import perf_counter

nltk.download("twitter_samples", quiet=True)
neg = twitter_samples.strings("negative_tweets.json")
pos = twitter_samples.strings("positive_tweets.json")
texts = neg + pos
labels = [0] * len(neg) + [1] * len(pos)
train_ids, test_ids = train_test_split(
    range(len(texts)), test_size=0.20, random_state=42, stratify=labels
)
TOKEN_PATTERN = r"(?u)\b[a-zA-Z][a-zA-Z]+\b"
token_re = re.compile(TOKEN_PATTERN)
stemmer = PorterStemmer()

def stem_text(text):
    tokens = token_re.findall(text)
    return " ".join(stemmer.stem(token.lower()) for token in tokens)

for use_stemming in (False, True):
    start = perf_counter()
    train_texts = [texts[i] for i in train_ids]
```

```
    test_texts = [texts[i] for i in test_ids]
    if use_stemming:
        train_texts = [stem_text(t) for t in train_texts]
        test_texts = [stem_text(t) for t in test_texts]
    vectorizer = CountVectorizer(
        min_df=2, max_df=0.95, token_pattern=TOKEN_PATTERN
    )
    X_train = vectorizer.fit_transform(train_texts)
    X_test = vectorizer.transform(test_texts)
    model = LogisticRegression(max_iter=300, solver="liblinear", random_state=0)
    model.fit(X_train, [labels[i] for i in train_ids])
    predicted = model.predict(X_test)
    elapsed = perf_counter() - start
    print("stemmed" if use_stemming else "raw",
          "vocabulary=", len(vectorizer.vocabulary_),
          "accuracy=", round(accuracy_score(
              [labels[i] for i in test_ids], predicted), 4),
          "seconds=", round(elapsed, 3))
```

Executed output

```
raw      vocabulary=3968  accuracy=0.7625  seconds=0.218
stemmed  vocabulary=3525  accuracy=0.7615  seconds=2.547
```

The corrected pipeline reduces the measured vocabulary by 443 terms (11.2%) on this fixed 80/20 split. Accuracy changes from 76.25% to 76.15%, a decrease of 0.10 percentage points, and stemming takes longer because each token must be processed. The correction improves feature construction by actually normalizing words before counting; this run does not show an accuracy gain. Stemming should therefore be retained only if validation on the target task supports the trade-off.

Evaluation details: 10,000 balanced tweets, 5,000 per label; stratified split with random_state=42; CountVectorizer uses min_df=2 and max_df=0.95; timing includes tokenization/stemming, vectorization, model fit, and prediction, but excludes dataset download. Exact timings vary by machine.

Dataset and software: NLTK Twitter Samples, https://www.nltk.org/data.html; CountVectorizer documentation, https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html.